



BBM465 – INFORMATION SECURITY LAB.

ASSIGNMENT 1

23.10.2024

Group ID: 15

1. Part 1: Cipher Implementation

a) Caesar Cipher

- **encrypt_caesar(plaintext, shift):** This function shifts the alphabet by the number of positions specified by the shift entered in the terminal and performs the encryption by changing the plaintext according to the corresponding shifted letters.
- **decrypt_caesar(plaintext, shift):** This function decrypts a message that was encrypted using the Caesar cipher by shifting the letters of the alphabet based on the given shift value. To do this, we use the previously written `encrypt_caesar(plaintext, shift)` function. The shift value is recalculated by subtracting it from 26, which is the length of the English alphabet. This effectively reverses the encryption and decrypts the message.

b) Affine Cipher

- **encrypted_affine(plaintext, a, b):** This function checks if `a` and the number of letters in the alphabet are relatively prime (using the `are_relatively_prime` function). For each letter in the plaintext, if the letter exists in the alphabet, its index value is calculated. The affine encryption formula is applied to compute a new index. The resulting letter is encrypted as uppercase if it is originally

uppercase, or as lowercase if it is originally lowercase. Non-letter characters are left unchanged.

- **decrypted_affine(plaintext, a, b):** This function finds the modular inverse of a (using the `mod_inverse` function). For each letter in the text, if the letter exists in the alphabet, its index value is calculated. The decryption formula is applied to compute a new index. The resulting letter is decrypted as uppercase if it is originally uppercase, or as lowercase if it is originally lowercase. Non-letter characters are left unchanged.

c) Mono-alphabetic Substitution Cipher

- **encrypted_mono(plaintext, key):** This function matches the letters of the alphabet with the letters in the key by first finding the index of each letter in the alphabet and then accessing the same index in the key. As a result of this mapping, each letter in the plaintext is replaced according to this mapping to produce the encrypted text.
- **decrypted_mono(plaintext, key):** This function operates under the same logic as the `encrypt_mono` function; the difference is that this time the index is first found in the key, and then the corresponding letter from the alphabet is retrieved. This results in recovering the original text.

2. Part 2: Cryptanalysis

a) Caesar Cipher

- **break_caesar(ciphertext):** This function calls the `decrypt_caesar` function for all shift values from 1 to 25. It checks the number of valid words in each solution using the `count_valid_words` function. The solution with the highest word count is stored and saved to a file.

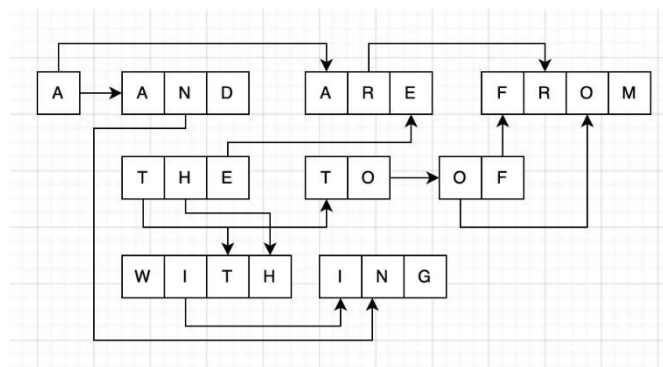
b) Affine Cipher

- **break_affine(ciphertext):** This function takes values of “ a ” between 1 and 25 and checks for values that have a greatest common divisor (gcd) of 1 with 26. For each “ a ”, it tries values of “ b ” from 0 to 25. It calls the `decrypt_affine` function from `ciphers.py` that we already defined and checks the number of valid words in each solution.
- **are_relatively_prime(a,b):** This function calculates whether two numbers are relatively prime using the Euclidean algorithm.
- **count_valid_words(text):** This function converts the text to lowercase, splits it into words, and checks if each word exists in the previously loaded set of words (dictionary).

c) Mono-alphabetic Cipher

- **break_mono(ciphertext):** This function decrypts a text encrypted with a mono-alphabetic substitution cipher. We use a dictionary-type variable called `decrypted_message`, which stores the mapping of each letter in the ciphertext to its corresponding letter in the English alphabet as we discover them. Punctuation marks and newline characters are removed when reading the encrypted text, and each word is stored in a list called `words`.

We base our frequency analysis on the standard letter frequency of the English language, which is "ETAOINSHRDLCLUMWFGYPBVKJXQZ". We assume that "E" is the most frequently used letter in any long text. Using this assumption, we try to identify as many letters as possible. Next, we identify "a" by finding the most common letter among single-letter words. After that, we look for three-letter words like "and" and "the", using certain limitations. For the word "from," if it doesn't appear in the text, we simply skip it. By this process, we aim to decrypt thirteen letters. As each letter is discovered, the corresponding variables are updated. The process order to find thirteen letter is below:



The letters that are not yet identified are listed as their equivalents in the English alphabet under `not_found_letters_alph = ['S', 'L', 'C', 'U', 'Y', 'P', 'B', 'V', 'K', 'J', 'X', 'Q', 'Z']`, and the corresponding encrypted letters are listed in `not_found_letters_text = ['L', 'S', 'X', 'E', 'N', 'H', 'W', 'C', 'A', 'B', 'P', 'J', 'M']` based on frequency analysis for example0.

The expression $(\text{if } \text{len}(\text{word}) * 0.8 \leq \text{count} < \text{len}(\text{word}))$ is used to verify that the word being checked contains enough known letters (at least 80%) while still leaving some letters unknown. Using the known letters, the encrypted word is partially decrypted, and any unknown letters are added to a list. In `not_found_letters_text`, we match the encrypted letters with their potential equivalents in `not_found_letters_alph`, and check if the decrypted word exists in `dictionary.txt`. If the frequencies don't match, we check the letters directly adjacent in the frequency order. If it still doesn't match, we check two positions to the left and right. We stop there, assuming that the letter frequency of a long text won't deviate significantly from the standard. This process continues until every letter in the word is known.

3. Resources

- <https://crypto.interactive-maths.com/frequency-analysis-breaking-the-code.html>
- <https://www3.nd.edu/~busiforc/handouts/cryptography/cryptography%20hints.html>
- <https://cs.wellesley.edu/~fturbak/codman/letterfreq.html>